A Design Framework of Heterogeneous Approximate DCIM-Based Accelerator for Energy-Efficient NN Processing

Kyeongho Lee, *Member, IEEE*, Hyeyeong Lee, *Graduate Student Member, IEEE*, and Jongsun Park, *Senior Member, IEEE*

***Abstract*— Static random-access memory (SRAM) based digital compute-in-memory (DCIM) provides error-resilient computation at the expense of considerable power overhead of adder tree. In recent works, DCIM macro based on approximate computing mitigates the adder tree overheads, however, it faces a trade-off between power and neural network (NN) accuracy. The trade-off becomes more complicated in array-level CIM architecture since output channels of NN model have different sensitivities to approximation errors. In this paper, we propose a heterogeneous approximate DCIM-based accelerator design framework that achieves a good energy-accuracy trade-off for a specific NN model. The framework includes three key features: 1) Evolutionary algorithm-based search finds cost-efficient approximation points by pruning the design space. 2) Genetic algorithm-based channel-wise mapping creates heterogeneous approximation methods that effectively reduce DCIM energy consumption while maintaining high accuracy. 3) A hardware generation strategy decides the number of DCIM macros and their sizes, resulting in an energy-efficient DCIM-based accelerator tailored for the given NN model. Experimental results show that employing the proposed heterogeneous channel-wise mapping significantly enhances the energy efficiency compared to a homogeneous mapping. Moreover, the proposed framework can produce heterogeneous DCIM-based accelerators that consume less energy than state-of-the-art approximate DCIM approaches.**

***Index Terms*— SRAM, compute-in-memory, CIM, heterogeneous approximation, DCIM generation.**

I. Introduction

DEEP neural networks (DNNs) are being widely utilized in various AI tasks such as natural language processing (NLP) [1] and computer vision (CV) [2] applications, due to their outstanding performance. This enhanced performance of DNNs comes at the cost of a large number of memory accesses, which limits the performance of accelerators based on Von Neumann architecture. To overcome the memory access problem, previous accelerator studies have focused on data precision reduction [3], [4], [5], model compression [6], [7], and memory mapping optimization [8], [9]. However, data movement cost remains a major concern.

As one of the research efforts to alleviate the memory access burden, compute-in-memory (CIM) is being actively studied. CIM paradigm aims to reduce data movement between the memory and computing units by performing computations directly within the memory. Recent studies on CIM [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26] have been conducted for various memory technologies, including static random-access memory (SRAM), resistive RAM (ReRAM), and magnetoresistive RAM (MRAM). Among the technologies, SRAM-based CIM has attracted attention as it is relatively simple to integrate within existing CMOS fabrication processes.

In the SRAM-based CIM, analog-domain CIM (ACIM) [10], [11], [12], [13], [14], [15] is widely employed to enhance both area and energy efficiencies. However, ACIMs are susceptible to computation errors due to process, voltage, and temperature (PVT) variations, which can lead to accuracy degradation. Recently, digital-domain CIM (DCIM) [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26] has been introduced as an error-resilient solution. Despite its advantage, the large power consumption of the adder tree in DCIM remains a significant challenge. To mitigate this problem, several DCIM studies [16], [17], [18] have employed approximate adders in the adder tree, compromising on accuracy.

In approximate computing research on DNNs, it is well-known that output channels within a network model show different sensitivities to errors (or error resilience) [27], [28], [29], [30]. Although the output channels have different sensitivities to computation errors, most of the previous DCIM works employ homogeneous DCIM macros in accelerator design. Our main idea in this work is that if heterogeneous DCIM macros (with non-uniform approximation levels) are employed considering the different error resilience of output channels, the hardware cost including energy and area could be efficiently traded off with DNN accuracies. However, combining channel-wise error sensitivity and non-uniform approximations significantly enlarges the design space, which

Received 20 August 2024; revised 4 December 2024; accepted 7 January 2025. Date of publication 24 January 2025; date of current version 30 July 2025. This work was supported in part by the National Research Foundation of Korea (NRF) grant funded by the Ministry of Information and Communications Technology (MSIT) under Grant RS2024-00345481 and Grant RS-2024-00405495; in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) grant funded by Korean Government (MSIT) under Grant 2022-0-00266 and Grant RS-202300229028; and in part by Samsung Electronics Company Ltd., under Grant IO201211-08087-01. This article was recommended by Associate Editor S. Sharma. *(Corresponding author: Jongsun Park.)*

Kyeongho Lee and Hyeyeong Lee are with the Library Development Team, Foundry Business of Samsung Electronics, Hwaseong-si 18448 South Korea (e-mail: keo2.lee@samsung.com; hy1503.lee@samsung.com).

Jongsun Park is with the School of Electrical Engineering, Korea University, Seoul 02841, South Korea (e-mail: jongsun@korea.ac.kr).

Digital Object Identifier 10.1109/TCSI.2025.3530637

1549-8328 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:30:07 UTC from IEEE Xplore. Restrictions apply.

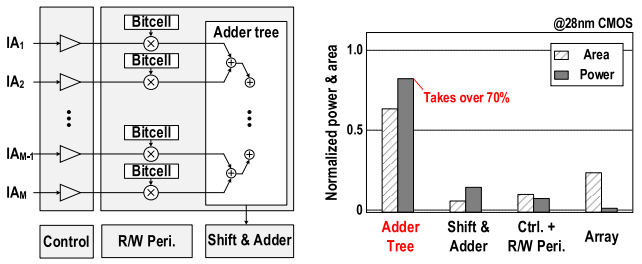


Fig. 1. Typical DCIM macro structure and its energy breakdown.

makes finding a good energy-accuracy trade-off points manually to be almost impractical.

In this paper, we present an automated design framework for heterogeneous approximate DCIM architecture that aims at energy-efficient NN processing. The proposed framework finds a good energy-accuracy trade-off design point for a given NN model. It also creates specific hardware for DCIM-based accelerator. The main contributions of this work can be summarized as follows:

1) This is the first work to introduce heterogeneous approximation as a solution for energy-efficient DCIM-based accelerators. In this work, we also propose an automated workflow for the accelerator design.
2) Evolutionary algorithm-based search (EAAS) and genetic algorithm-based channel-wise (GA-CHA) mapping approach have been proposed to create heterogeneous approximation methods that show good energy-accuracy trade-off points.
3) The proposed framework supports an automated hardware generation strategy to decide the number of DCIM macros and the macro sizes. Based on the proposed workflow, an energy-efficient DCIM-based accelerator is generated for a given NN model.

The rest of this article is organized as follows. Section II presents backgrounds and related works. Section III introduces the overall workflow of the proposed framework. Section IV explains three main design methodologies in the framework. Experimental results are presented in Section V. Finally, conclusions are drawn in Section VI.

II. Preliminaries

A. Approximate DCIM Macro and Related Works

Fig. 1 illustrates the architecture of a typical DCIM macro, where the multiplication of inputs and weights is performed directly within the bitcell array, and the partial products are accumulated using an adder tree. This structure enables competitive energy efficiency in DCIMs. However, the adder tree, which accounts for over 70% of the total power consumption in DCIMs, poses a significant challenge due to its high power usage. To reduce power consumption, several studies have proposed using approximate adders in the adder tree, trading off accuracy for energy savings.

Fig. 2 illustrates an example of approximate DCIM macro structure, where an approximate adder is implemented between two adjacent rows. This adder performs an approximation that reduces the input bit-width of adder tree. Subsequently, the partial products are accumulated using a full-adder-based

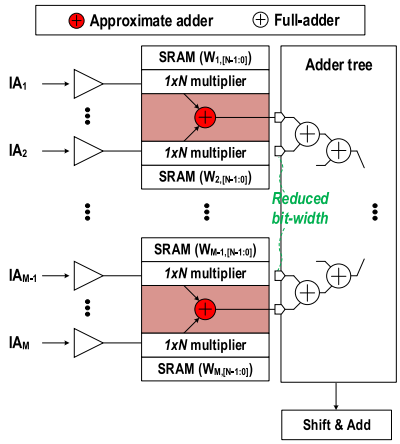


Fig. 2. Approximate DCIM macro structure.

adder tree, producing an approximate partial sum. This simplification of the adder tree significantly reduces the area and power consumption of DCIM macro.

Several previous approaches [16], [17] have focused on designing the approximate adder at the circuit level. In DIMC [16], the approximate adder is implemented using AND/OR gates. This configuration reduces the area and power consumption associated with the adder tree but results in significant accuracy loss. In LSAC [17], a low-error approximation is used to calculate higher-bit positions of weights, while a high-error approximation is used for lower-bit positions. By considering the bit-wise error sensitivity of weights, LSAC reduces the power and area while maintaining high accuracy. Unfortunately, the approaches do not attempt to find an optimal approximation, resulting in poor accuracy.

To address the accuracy issue, in BASER [18], six bit-wise approximate compressors are presented, and a search algorithm is proposed to decide a low-power approximation scheme for a given quantization bit-width. However, these previous studies apply approximations uniformly across the entire network, without considering the layer-wise or channel-wise error sensitivity. As a result, significant accuracy degradation may occur when processing complex NN workloads, such as ResNet-18 on ImageNet.

In this work, we adopt a heterogeneous approximation for DCIM-based accelerator design, addressing the channel-wise error-sensitivity of NNs. We also develop search algorithms to find high-quality solutions within the framework's expansive design space.

B. Basic of Genetic Algorithm

As the search algorithms used in this work are based on genetic algorithm (GA), the fundamentals of GA will be briefly explained in this subsection.

Genetic algorithm (GA) is widely used in search and optimization problems. The algorithm begins by encoding potential solutions into strings of genes (known as chromosomes). Initially, a population of the chromosomes is randomly generated and then evaluated with a fitness function. Individuals with low fitness values are often eliminated, and those with high fitness values are used to produce a new population through genetic operators (selection, crossover, and

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:30:07 UTC from IEEE Xplore. Restrictions apply.

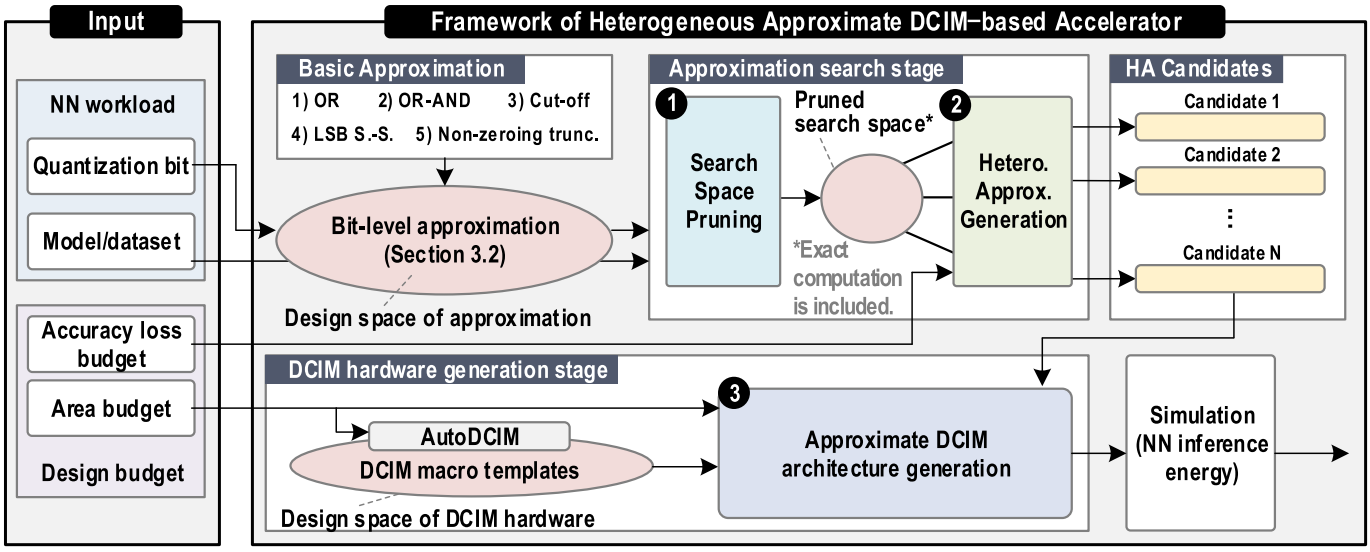


Fig. 3. Overview of the framework.

mutation). By iteratively applying the operators, the population evolves into better solutions. The process continues until a termination condition is satisfied, such as a satisfactory fitness level or a maximum number of generations.

III. Framework Overview

A. Flow of the Framework

The framework's problem can be formulated as follows: "*Given the NN workload, NN accuracy loss budget, and area budget as inputs, find an energy-efficient DCIM-based accelerator using heterogeneous approximation.*"

Fig. 3 shows the framework overview. First, inputs are provided and then the design space for approximation and DCIM hardware is constructed. Through the approximation search stage, heterogeneous approximation (HA) candidates are created. Next, in the hardware generation stage, DCIM-based accelerators are generated based on the HA candidates. Finally, the accelerators are evaluated using consumed energy during inference. As the framework's output, an accelerator that achieves minimum energy consumption is selected.

In the framework, five basic approximations are embedded, as shown in Fig. 3. The basic approximations are combined with a given quantization bit-width ($\boldsymbol{B_W}$) to generate tens or hundreds of different approximation methods. Details will be discussed in Section III-B. The design space of DCIM hardware contains various DCIM macros generated through AutoDCIM [31], including area costs. The design flow of framework consists of three main steps: search space pruning, heterogeneous approximation generation, and DCIM architecture design. In the search space pruning step (❶), the design space of approximation is pruned by EAAS to find cost-efficient approximations. This will be introduced in Section IV-A. Based on the pruned search space, the heterogeneous approximation generation step (❷) creates HA candidates that meet the given accuracy loss constraint using GA-CHA mapping. This will be introduced in Section IV-B. In the approximate DCIM architecture generation step (❸), it takes user-defined area budget and one of the HA candidates as input. Then, under a given area budget, design parameters of approximate DCIM architecture (i.e., row/column sizes and macro counts) are specified. This will be introduced in Section IV-C.

B. Design Space of the Framework

This section will introduce the design space of approximation. It includes five basic approximations, and how to create the bit-level approximations using the basic approximations. The advantage of investigating bit-level approximations will be also explained.

Basic Approximation: For explanation, 8-bit operand is used, and power and area reductions are evaluated compared to an 8-bit full-adder (FA) based on synthesis results using 28 nm CMOS technology. The root mean square error (RMSE) is used as a metric for approximation error.

1) OR-Based Addition: It performs a bitwise OR operation. The use of simple logic gates reduces power by 87% and area by 88%. However, the largest approximation errors (RMSE = 45.1) are induced.

2) OR-AND-Based Addition: It combines OR-based addition with a bitwise AND operation. For example, when adding two unsigned 4-bit operands, the bitwise OR operation is executed across all bits, while the bitwise AND operation is exclusively performed on the most significant bit (MSB). If the AND operation output is '1', the output is set to '$1111_{(2)}$'. If the result is '0', the output remains the same as the OR-based addition. The OR-AND-based method achieves 83% reduction in power, 82% reduction in area, and has a 39.6 RMSE.

3) Cutoff-Based Addition: For the addition of two unsigned N-bit operands, this approximation sets the output's upper limit to 2^N-1. With this approach, power and area are reduced by 38% and 27%, respectively, and the RMSE reaches 36.9.

4) LSB Skip-Shift-Based Addition: In this approach, the least significant bit (LSB) of the parameter is omitted from

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:30:07 UTC from IEEE Xplore. Restrictions apply.

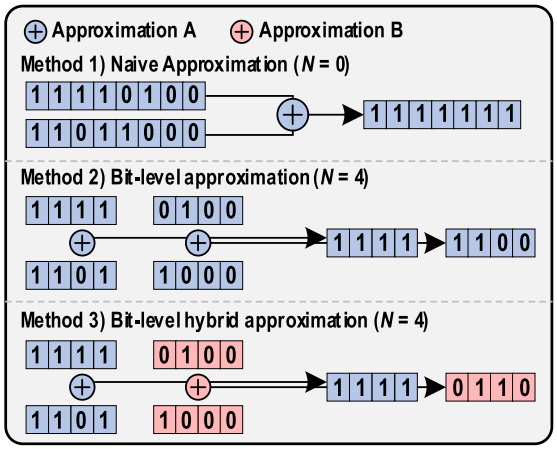


Fig. 4. Three methods to create the bit-level approximation.

the computation. Some of the introduced errors in the output are corrected through a 1-bit shifting operation. For example, when adding $1011_{(2)}$ (11 in decimal) and $1101_{(2)}$ (13 in decimal), $101_{(2)}$ and $110_{(2)}$ are used for the addition. Then, the result of $1011_{(2)}$ becomes $10110_{(2)}$ (22 in decimal) after the 1-bit shifting operation. In DCIM macro featuring LSB Skip-Shift-based addition, the 1-bit shift is applied to the adder tree's output. The approach achieves an RMSE value of 1.2, while power and area are reduced by 27% and 18%, respectively.

5) Non-Zeroing Bit Truncation-Based Addition: [37] introduces an approximation technique called Non-zeroing Bit Truncation (NZBT). The method selectively retains or modifies certain bits to enhance computational accuracy while reducing power consumption. To understand its operation, consider the addition of two 8-bit numbers: A = $10111001_{(2)}$ (185 in decimal) and B = $01110110_{(2)}$(118 in decimal). In this method, only the most significant 1 in the truncated portion is preserved, while the remaining bits are set to 0. For example, the 4 LSBs of A ($1001_{(2)}$) are approximated as $1000_{(2)}$, and those of B ($0110_{(2)}$) are reduced to $0100_{(2)}$. As a result, the approximate sum becomes 300, with an error of just 3 compared to the exact result 303.

Design Space of Approximation: As shown in Fig. 4, when the input is inserted, five basic approximations expand the bit-level approximations to construct design space of approximation. Fig. 4 presents three ways to create the bit-level approximation by using the basic methods. For example, a single basic approximation can be applied to the entire bits (naï ve approximation of Fig. 4) or split-and-applied to ($\boldsymbol{B_W}$-N)|N bits (bit-level approximation of Fig. 4). Take an 8-bit $\boldsymbol{B_W}$ and the Cutoff-based addition for $N = 0$ and 4 as an example. Adding $1111_0100_{(2)}$ to $1101_1000_{(2)}$ results in $1111_1111_{(2)}$ for $N = 0$, and $1111_1100_{(2)}$ for $N = 4$. In another way, different basic methods can be split-and-applied to ($\boldsymbol{B_W}$-N)|N bits (bit-level hybrid approximation of Fig. 4). Here, by considering hardware complexity, the bit-level approximation is limited to the case where $\boldsymbol{B_W}$ is split into two parts (refer to Fig. 4).

In the expanded design space, the magnitude of the approximation error becomes more diverse than in the basic methods, increasing the likelihood of finding better solutions for heterogeneous approximation.

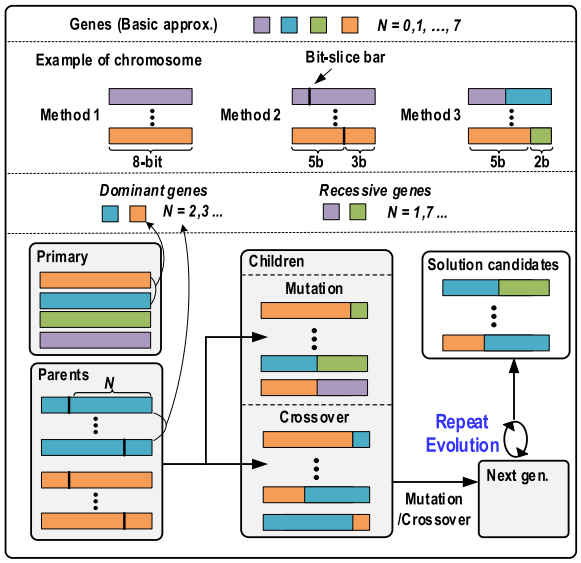


Fig. 5. Evolutionary algorithm-based approximation search (EAAS).

IV. Framework Design Strategies

This section will introduce three design strategies of the framework. To explain the strategy, a case study of 8-bit quantized ResNet-18 is used. Although ResNet-18 is presented as a case study in Section IV, the proposed framework can be used in other network models with various quantization bits, such as 6-bit quantized VGG-16.

A. Search Space Pruning

1) Problem: Exploring bit-level approximations provides a higher chance of finding better solutions. However, the bit-level approximation needs retraining efforts to mitigate accuracy degradation caused by approximation. The retraining process demands extensive resources and consumes tens to hundreds of GPU/CPU hours [32], [33], [34]. So, investigating all bit-level approximations poses a substantial challenge. To tackle the problem, we develop a novel evolutionary algorithm-based approximation search (EAAS).

2) Method: Fig. 5 illustrates the overall procedure of EAAS. The main goal of the search is to find cost-effective approximation methods suitable for creating heterogeneity. Parameter settings are defined as follows: Five basic approximations and a variable N are designated as *genes*. Using these genes, the three approximation types shown in Fig. 4 are represented as chromosomes. In Fig. 5, the *bit-slice bar* indicates the N value for Methods 2 and 3. The fitness function employs a fine-tuned accuracy simulation, applying an approximation uniformly across the entire network. Among candidates with similar levels of accuracy degradation, those with greater energy savings are assigned higher fitness, while those with lower energy savings are assigned lower fitness.

The evolution process is as follows: Initially, the fine-tuned accuracy simulation is conducted on five primary cases of Method 1. Based on the results, the two cases showing higher

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:30:07 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EAAS DECISION MAKING FOR 8-BIT QUANTIZED RESNET-18 ON CIFAR-100

Approximation type	N value	Top-1 Accuracy loss	CIM power reduction	CIM area reduction
B_W–N: basic 3 N: basic 1	3	1.52	18.8 %	27.3 %
	4	**1.82**	**20.4 %**	**29.5 %**
	5	2.45	21.3 %	32.6 %
	6	**2.97**	**25.8 %**	**34.0 %**
B_W–N: basic 4 N: basic 2	**3**	**0.88**	**18.8 %**	**21.3 %**
	4	1.02	20.4 %	23.5 %
	5	1.14	21.3 %	25.6 %
	6	**1.31**	**25.8 %**	**27.0 %**
B_W–N: basic 4 N: basic 3	3	0.41	20.3 %	13.5 %
	4	0.29	22.4 %	14.0 %
	5	**0.44**	**23.7 %**	**15.2 %**
	6	0.71	25.2 %	16.9 %

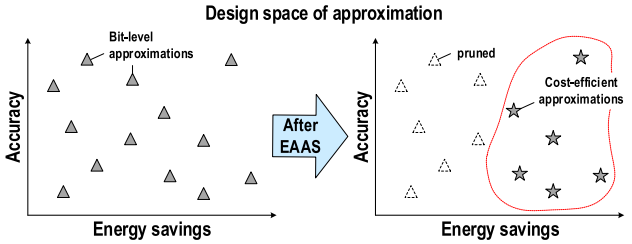


Fig. 6. Impact of EAAS on Bit-Level approximation design space.

accuracies are considered *dominant genes*, while the others are considered *recessive genes*. Subsequently, parent candidates are generated by varying N values in the case with *dominant genes*, such that the candidates correspond to Method 2 in Fig. 4. Accuracy simulations are then performed on the parent candidates to determine the dominant genes for the N value. Following this, crossover and mutation processes are carried out to produce children. The crossover is conducted between *dominant genes*, whereas the mutation involves both *dominant* and *recessive genes*. After that, the child candidates undergo a fitness evaluation. High-fitness cases evolve into the next generation, while low-fitness cases are eliminated from design space. The process is repeated until the crossover/mutation cases are fully explored. During this evolution process, solution candidates are decided as cases that achieve higher energy savings at a similar level of accuracy loss. By employing evolutionary search, the proposed algorithm constructs a pruned search space by selectively exploring design space of bit-level approximation.

Table I shows examples of EAAS decision making for 8-bit quantized ResNet-18 on CIFAR-100. EAAS evaluates candidates based on their Top-1 accuracy loss and CIM power and area reductions. Specifically, candidates are categorized into 1% accuracy loss ranges to systematically assess their trade-offs. Within each accuracy loss range, the algorithm selects the candidate that achieves greater power reduction. For example, when combining basic approximation methods (1) and (3), in the 1–2% accuracy loss range, the N =4 configuration results in a 1.82% accuracy loss while attaining a 20.4% power reduction and a 29.5% area reduction, leading it to be considered as a survivor. Similarly, within the 2–3% accuracy loss range, the N =6 configuration incurs a 2.97% accuracy loss but offers substantial energy savings with a 25.8% power reduction and a 34.0% area reduction. This selective process ensures that, for each 1% step of accuracy degradation, the most energy-efficient approximation methods are retained.

Fig. 6 illustrates the design space of bit-level approximations for both cases, with and without applying EAAS. By considering the impact of approximations on CIM power and area, the search space size is reduced by 77% for the example case.

B. Heterogeneous Approximation Generation

1) Problem: As mentioned in Section I, output channels have different sensitivities to approximation errors [27], [28], [29], [30]. So, when approximations are differently applied to output channels, i.e., if heterogeneous approximation is applied, hardware cost can be reduced while maintaining high accuracy.

The straightforward way of implementing channel-wise approximation is to analyze the sensitivity of all output channels and apply approximations adaptively based on the analysis, such as [27]. In the approach, channels are grouped based on channel sensitivity criteria, and then approximations are decided. However, as the criteria depend on hyperparameters, this approach encounters difficulty supporting diverse NN workloads, making it difficult to establish a design automation process. To address this concern, we use a GA-based channel-wise approximation (GA-CHA).

2) Method: As shown in Fig. 3, based on the pruned search space, the heterogeneous approximation search is conducted by GA-CHA. In the search space, exact computation is also included to create a sophisticated approximation method. GA-CHA takes one approximation method and accuracy loss constraint (i.e., accuracy budget) as input. Through the algorithm, output channels to assign the approximation (i.e., *channel-wise mapping*) are decided. The process is repeated using various approximations to generate candidates satisfying the given accuracy condition.

Before explaining the algorithm, we introduce two policies of GA-CHA. First, most channels may be assigned to a particular single approximation, resulting in homogeneous approximation. To prevent this, we limit the number of channels assigned to one approximation (represented as *channel group*). Second, note that the number of channels is different depending on the layer. If the ratio between *channel groups* varies across layer, approximate DCIM-based accelerator will face inefficient dataflow problem such as load imbalance. To prevent this, we maintain a consistent ratio between *channel groups* throughout all layers.

Algorithm 1 presents the GA-based channel-wise approximation. We first define an output channel assigned to given approximation as genetic representation. Then, we create an initial population that contains 100 individuals for each NN layer. Next, we conduct a fitness evaluation using accuracy simulation. The fitness results are ranked according to the magnitude of the loss (**line 8**). From the result, the 40 individuals (*sortedPop*) with the smallest loss survive while the rest

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:30:07 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 The Genetic Algorithm for Channel-Wise Approximation

Input: Accuracy loss constraint as *AccLoss*, a solution of previous GA as *previousIndiv*
Output: Channel-wise approximation strategy as *TheBestIndividual*

```
for i = 1 to 200 do
    if i = 0 then
        for j = 1, 2, ..., number_of_conv_layer do
            currentPop ⟵ Initial_Population(previousIndiv)
        end for
    else
        for j = 1, 2, ..., number_of_conv_layer do
            sortedPop ⟵ SortbyFitnessEval(currentPop)
            if top-5 cases of sortedPop do not satisfy AccLoss
then
                Terminate the algorithm
            else then
                { child_1, ..., child_5} ⟵ Selection(sortedPop)
                {child_6, ..., child_45} ⟵ Crossover(sortedPop)
                {child_46, ..., child_100} ⟵ RandomGeneration()
                currentPop ⟵ {child_1, ..., child_100}
        end for
    end for
TheBestIndividual ⟵ Select the best accuracy in FitnessEval (currentPop)
```

Return: *TheBestIndividual*

are eliminated. Subsequently, the next generation is generated through genetic operators: *selection*, *crossover*, and *random generation*.

In the *selection* (**line 12**), the top-5 individuals are passed on to the next generation with the genetic representation preserved. In the *crossover* (**line 13**), two individuals are selected from the 40 survivors (*sortedPop*). Then, the genetic information is extracted from the two individuals and crossed to create a new individual. Based on this, 40 individuals are newly produced for the next generation. The remaining 55 individuals are randomly drawn from the initial population (**line 14**) to prevent the premature convergence of the algorithm. The evolution is repeated 200 times, and the individual with the smallest loss is considered the solution of a given approximation (i.e., the mapping strategy between channel and approximation). Please note that an independent population is created and processed for each NN layer. This separate evolution for populations allows the algorithm to consider the layer-wise error-sensitivity as well.

To complete the HA candidate generation, GA-CHA continues iteratively using various approximation methods. Between iterations of the GA, the previous solution is fed into the next GA (**line 4**). To terminate candidate generation early that fails to satisfy the accuracy loss constraint, we preferentially run the algorithm for cases where the approximation error is larger. Additionally, each GA-CHA iteration involves retraining the model with the applied approximation method, ensuring that the performance loss is accurately evaluated and the approximation strategy is refined during the process.

C. Approximate DCIM Architecture Design

As shown in Fig. 3, the framework embeds AutoDCIM [31], an automated DCIM macro generation tool for generating

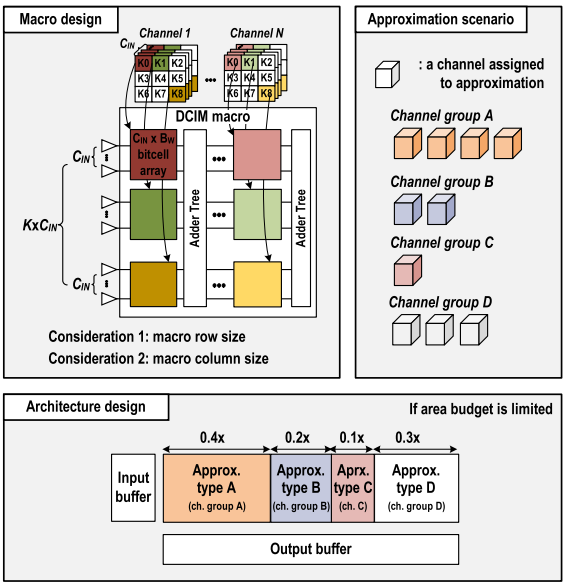


Fig. 7. Design considerations of heterogeneous approximate DCIM-based ac-celerator.

circuits and layout of SRAM-based DCIM macro. By adopting the tool, the framework creates diverse DCIM macro templates with area costs. To complete creating DCIM-based accelerator, the framework focuses on deciding design parameters of approximate DCIM architecture, such as macro row/column size and the number of CIM macros.

1) Base Architecture: It is important to note that SRAM CIM-based accelerators have more severe limitations in storage capacity than other memory technologies in area-constrained environments. So, holding parameters of multi-layers is challenging within the same operation cycle. For this reason, previous SRAM CIM-based accelerators [35], [36] have employed single-layer parallel architecture. The framework also utilizes the architecture.

2) Design Rules: Fig. 7 shows design considerations of heterogeneous approximate DCIM architecture within a convolution layer. As shown in the upper-left part, input channel size ($\boldsymbol{C_{IN}}$) and kernel size ($\boldsymbol{K}$) are considered when deciding DCIM macro row size. Each local array is implemented by $\boldsymbol{C_{IN}} \times B_W$ size, and $\boldsymbol{K}$ number of local arrays complete the row size. So, the macro row size can be expressed as $\boldsymbol{C_{IN}} \times \boldsymbol{K}$. The basic mapping method for the DCIM macro is that components of arbitrary channel (*Channel N*) are shared along the same column, and data from different channels are stored on a column-by-column manner. Each DCIM macro involves a single approximation type, and the DCIM architecture employs the single-layer parallel processing. Given these conditions, the focus of the framework is on achieving an efficient design within the limitations of available area.

If area budget is sufficient, it is desirable that the approximate DCIM macro size is decided based on the allocated parameter size. But, if the area budget is limited, it is important to consider the ratio between *channel groups*. As shown in the upper-left part of Fig. 7, output channels are mapped in the column direction of CIM macro. So, each approximate DCIM column size is decided by considering the ratio between *channel groups* (refer to the rest parts of Fig. 7). In deciding

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:30:07 UTC from IEEE Xplore. Restrictions apply.

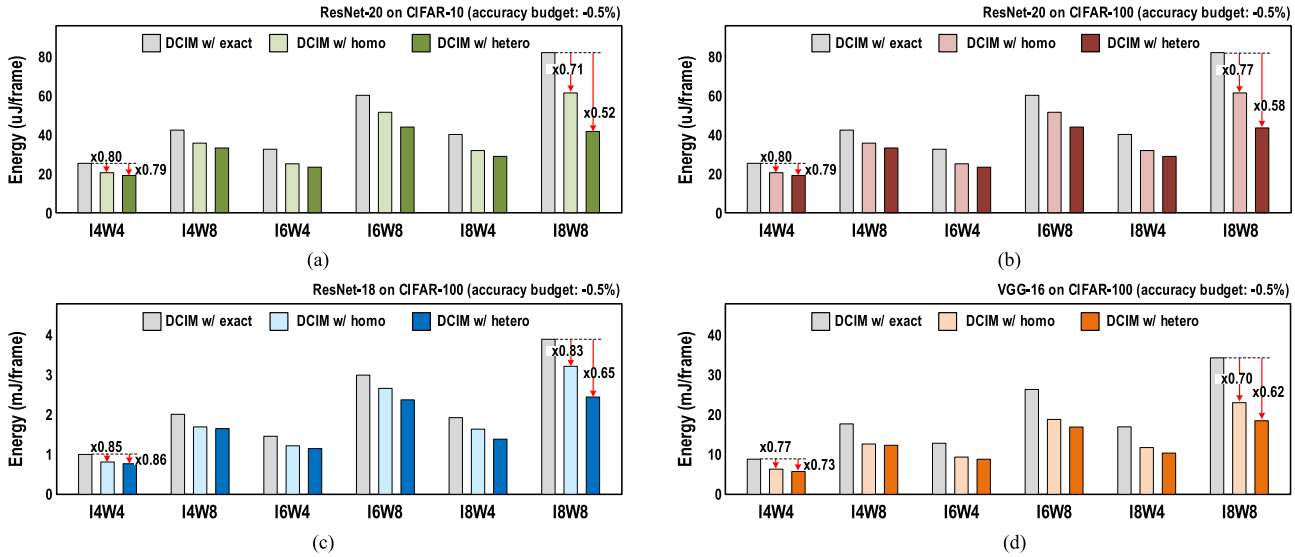


Fig. 8. Consumed energy of DCIM-based accelerators (Joules per frame) with different quantization bits. (a) ResNet-20 on CIFAR-10. (b) ResNet-20 on CIFAR- 100. (c) ResNet-18 on CIFAR-100. (d) VGG-16 on CIFAR-100.

DCIM macro counts in the architecture, the counts are decided based on approximation types counts in an HA candidate if the area budget is limited. On the other hand, if the area is sufficient, the macro counts are duplicated to exploit weight duplication technique. Based on these design rules, the hardware generation strategy creates an energy-efficient DCIM architecture for given heterogeneous approximation.

V. Experiment

A. Experimental Setup

To assess the impact of approximation on accuracy, ResNet-18, ResNet-20, ResNet-50, VGG-16 and MobileNet-v2 architectures are tested using the CIFAR-10, CIFAR-100, Tiny-ImageNet and ImageNet datasets. The maximum acceptable accuracy loss is set differently based on the dataset's complexity: 1% for CIFAR-10/100, and 2% for Tiny-ImageNet. Area budgets are estimated through manual design, where a 36KB DCIM architecture using exact computation occupies 1.8mm^2 silicon area at 28nm CMOS technology. The energy consumption of CIM-based NN processing is calculated based on SPICE simulation.

Homogeneous Approximate DCIM Baseline: For comparison, we need to implement homogeneous approximate DCIM architecture that satisfies accuracy loss budget. To identify the best homogeneous approximation method, we modify the EAAS algorithm so that the fitness function only considers the accuracy loss budget. Among the solution candidates generated by the modified EAAS, the approximation that achieves the greatest energy reduction is selected. The DCIM architecture for this homogeneous approximation is generated using DCIM generation tool within the framework. The architecture is denoted as *DCIM with homo* in the following subsection. A heterogeneous approximate DCIM architecture produced by the framework is denoted as *DCIM with hetero*. By comparing the two architectures, we will explore the energy-accuracy trade-offs under different design budgets in the next subsection.

B. Experimental Results

1) Case Studies With Different Quantization Bits: Fig. 8 shows the consumed energies of DCIM-based accelerators that are obtained by inserting different quantization bits into the framework. In the figure, the term *DCIM with exact* denotes an architecture consisting of DCIM macros without applying any approximation methods. In Fig. 8(a), for ResNet-20 on CIFAR-10 using 4-bit activation and 4-bit weight (I4W4), the energy consumption of *DCIM with homo* is reduced to 0.80×, and that of *DCIM with hetero* is reduced to 0.79×, compared to *DCIM with exact*. When employing 8-bit activations and weights (I8W8), *DCIM with homo* reduces the energy consumption by 0.71×, while *DCIM with hetero* can achieve the reduction up to 0.52×. In Fig. 8(b) and Fig. 8(c), the result for ResNet-20 on CIFAR-100 and ResNet-18 on CIFAR-100, respectively, show that the difference in energy reduction between *DCIM with homo* and *hetero* becomes larger as the quantization bits increases. Fig. 8(d) shows the results for VGG-16 on CIFAR-100. It also shows a similar trend, where the energy savings difference between the homogeneous and heterogeneous DCIM methods becomes bigger as the quantization bit size increases. Consequently, the experiments demonstrate that at larger quantization bit sizes, heterogeneous approximate DCIM is more effective in achieving good energy-accuracy trade-offs than homogeneous approach.

2) Impact of Accuracy Loss Constraints: Fig. 9 presents the consumed energies of *DCIM with homo* and *hetero* for various models and datasets (all 8-bit quantized), each with different accuracy budgets. In the figure, we calculate the energy reductions between two different accuracy budgets for each DCIM and compare the results to show which approximation is more effective. In Fig. 9(a), when comparing the results for ResNet-20 on CIFAR-10 under accuracy budgets of −0.5% and −1.0%, energy consumption of *DCIM with homo* decreases by 0.77×, while that of *DCIM with hetero* is reduced by 0.53×. Fig. 9(b) shows that for ResNet-20 on

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:30:07 UTC from IEEE Xplore. Restrictions apply.

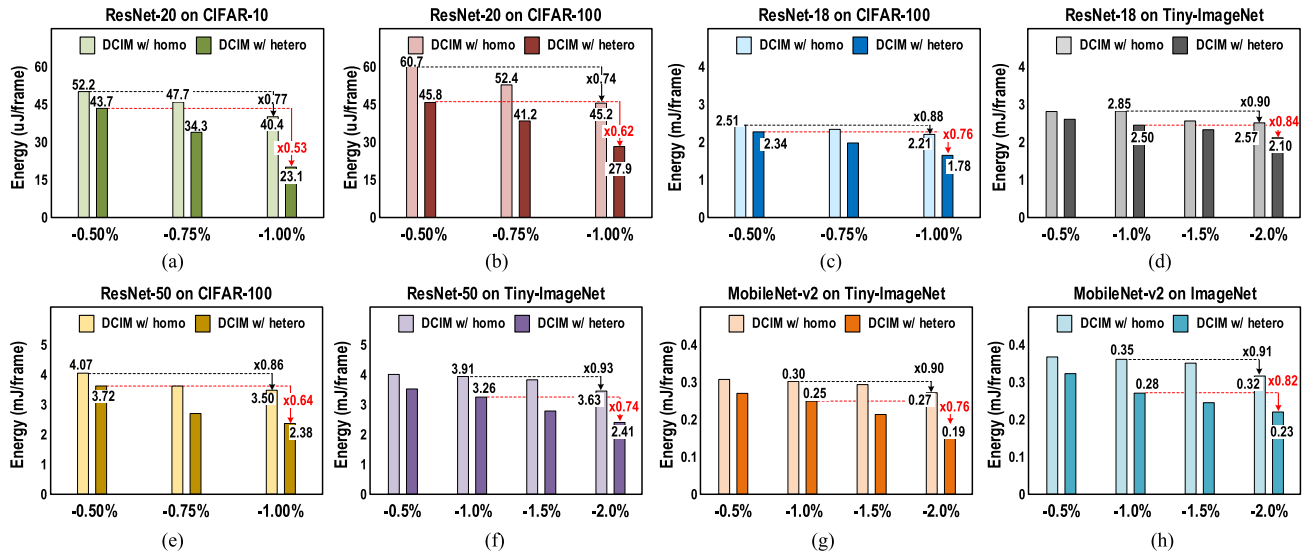


Fig. 9. Consumed energy of DCIM-based accelerators (Joules per frame) with different accuracy budgets. (a) ResNet-20 on CIFAR-10. (b) ResNet-20 on CIFAR- 100. (c) ResNet-18 on CIFAR-100. (d) ResNet-18 on Tiny-ImageNet. (e) ResNet-50 on CIFAR-100. (f) ResNet-50 on Tiny-ImageNet. (g) MobileNet-v2 on Tiny- ImageNet. (h) MobileNet-v2 on ImageNet.

TABLE II

COMPARISON WITH OTHER HOMOGENEOUS APPROXIMATE DCIM WORKS

NN workload	Approximation method	Top-1 Accuracy loss	Delay reduction	Power reduction	Energy reduction	Area reduction
ResNet-20 on CIFAR-10	[16]	0.83	22.8 %	34.3 %	49.3 %	31.5 %
	[17]	0.40	6.7 %	19.0 %	24.4 %	16.2 %
	[38]	0.29	9.4 %	9.7 %	18.2 %	11.4 %
	Hetero. (our)	**0.44**	**19.4 %**	**33.2 %**	**46.2 %**	**26.9 %**
ResNet-20 on CIFAR-100	[16]	2.51	22.8 %	34.3 %	49.3 %	31.5 %
	[17]	1.27	6.7 %	19.0 %	24.4 %	16.2 %
	[38]	0.47	9.4 %	9.7 %	18.2 %	11.4 %
	Hetero. (our)	**0.96**	**17.5 %**	**28.1 %**	**40.7 %**	**23.5 %**
ResNet-18 on CIFAR-100	[16]	2.15	22.5 %	34.7 %	49.4 %	31.5 %
	[17]	1.10	6.4 %	19.2 %	24.4 %	16.2 %
	[38]	0.42	9.1 %	10.0 %	18.2 %	11.4 %
	Hetero. (our)	**0.99**	**18.2 %**	**28.1 %**	**41.5 %**	**22.7 %**
ResNet-18 on Tiny-ImageNet	[16]	9.72	22.5 %	34.7 %	49.4 %	31.5 %
	[17]	6.31	6.4 %	19.2 %	24.4 %	16.2 %
	[38]	1.41	9.1 %	10.0 %	18.2 %	11.4 %
	Hetero. (our)	**0.98**	**15.8 %**	**20.7 %**	**33.2 %**	**15.7 %**
ResNet-50 on CIFAR-100	[16]	1.92	22.2 %	35.0 %	49.4 %	31.5 %
	[17]	1.04	6.0 %	19.5 %	24.3 %	16.2 %
	[38]	0.39	8.8 %	10.3 %	18.2 %	11.4 %
	Hetero. (our)	**1.00**	**17.6 %**	**30.6 %**	**42.8 %**	**23.9 %**
ResNet-50 on Tiny-ImageNet	[16]	8.24	22.2 %	35.0 %	49.4 %	31.5 %
	[17]	5.12	6.0 %	19.5 %	24.3 %	16.2 %
	[38]	1.15	8.8 %	10.3 %	18.2 %	11.4 %
	Hetero. (our)	**0.97**	**15.4 %**	**25.7 %**	**37.1 %**	**13.9 %**

CIFAR-100, energy consumption reduces to 0.74× with the *homo* approach while to 0.62× with the *hetero* one. The energy reductions on the relatively more complex CIFAR-100 dataset are less substantial, but heterogeneous DCIM still achieves better performance compared to its homogeneous counterpart.

Fig. 9 (c) and (d) show the results for ResNet-18 on CIFAR-100 and Tiny-ImageNet, respectively, while Fig. 9 (e) and (f) present the results for ResNet-50 on the same datasets. For the CIFAR-100 dataset results, the ResNet-50 model achieves a 0.64× reduction in energy consumption, whereas the ResNet-18 model reduces energy consumption by 0.76×. Similarly, on the Tiny-ImageNet dataset, the ResNet-50 and ResNet-18 models reduce energy consumption by 0.74× and 0.84×, respectively. As the NN model size increases, heterogeneous approximation can leverage more varied error-sensitivities, resulting in a greater energy reduction.

Fig. 9 (g) and (h) show the results for MobileNet-v2 on the Tiny-ImageNet and ImageNet datasets, respectively. On the Tiny-ImageNet dataset, the MobileNet-v2 model achieves a 0.76× reduction in energy consumption, while

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:30:07 UTC from IEEE Xplore. Restrictions apply.

on the ImageNet dataset, it achieves a 0.82× reduction. The results demonstrate that the framework effectively extends to MobileNet architectures. Despite the unique characteristics of MobileNet-v2, such as the use of depthwise separable convolutions, our approach continues to adapt effectively by focusing on layer-specific sensitivities.

From the results of Fig. 9, it is obvious that when processing the relatively complex dataset, *DCIM with hetero* reduces energy consumption more effectively than *DCIM with homo*. This is because more channels are assigned to approximations with large errors (i.e., more channels are processed by low-power DCIM) when the accuracy budget increases.

C. Comparison With Previous Works

Table II shows the energy-accuracy trade-offs between heterogeneous approximate DCIM and other homogeneous approximate DCIM works [16], [17], and [38]. Each previous approach uses a different quantization bit, so for comparison, previous approximations are modeled to fit 8-bit precision. The reductions shown in the table are calculated by comparing the approximate methods with exact computation (i.e., without any approximation).

As shown in the table, for the CIFAR-10/100 results (which are less complex), our heterogeneous approach achieves better energy reductions than [17] and [38] while maintaining comparable levels of accuracy loss. Specifically, [38] achieves an energy reduction of only 18.2% on both CIFAR-10 and CIFAR-100, which is significantly lower than the 46.2% and 40.7% achieved by our method. This shows that while [38] is effective at preserving accuracy, it compromises heavily on energy efficiency.

For the more complex Tiny-ImageNet dataset, homogeneous approximations face significant accuracy degradations, while [38] manages to preserve accuracy with a loss of 1.41%. However, this comes with only 18.2% energy reduction. In contrast, our heterogeneous approach effectively reduces energy consumption while maintaining an accuracy loss of less than 1%.

By achieving both meaningful energy reductions and high accuracy, our heterogeneous approach provides a more practical solution for energy-efficient AI systems compared to homogeneous approaches.

VI. Conclusion

This paper proposes a design framework for heterogeneous approximate DCIM-based accelerator. In the framework, our evolutionary algorithm-based approximation search (EAAS) reduces the vast design space of bit-level approximation, improving the overall search efficiency. The genetic algorithm-based channel-wise (GA-CHA) mapping decides the allocation of output channels to approximations, achieving good energy-accuracy trade-off points. In addition, the DCIM hardware generation strategy creates energy-efficient accelerator that satisfies the user-defined design budgets. The results show that employing heterogeneous DCIM macros is a good solution for energy-efficient design with high accuracy. At a similar level of accuracy loss, our approach consumes less energy compared to other approximate DCIM works.

Acknowledgment

The EDA tool was supported by the IC Design Education Center (IDEC), South Korea.

References

[1] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 6000–6010, doi: 10.5555/3295222.3295349.

[2] M. Choi, H. Kim, B. Han, N. Xu, and K. M. Lee, "Channel attention is all you need for video frame interpolation," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Apr. 2020, vol. 34, no. 7, pp. 10663–10671, doi: 10.1609/aaai.v34i07.6693.

[3] Y. Yuan, C. Chen, X. Hu, and S. Peng, "EvoQ: Mixed precision quantization of DNNs via sensitivity guided evolutionary search," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2020, pp. 1–8, doi: 10.1109/IJCNN48605.2020.9207413.

[4] A. Jiang, L. Du, and Y. Du, "GroupQ: Group-wise quantization with multi-objective optimization for CNN accelerators," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 7, pp. 2071–2083, Jul. 2024, doi: 10.1109/TCAD.2024.3363073.

[5] A. Capotondi, M. Rusci, M. Fariselli, and L. Benini, "CMix-NN: Mixed low-precision CNN library for memory-constrained edge devices," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 5, pp. 871–875, May 2020, doi: 10.1109/TCSII.2020.2983648.

[6] X. Ma et al., "Non-structured DNN weight pruning—Is it beneficial in any platform?" *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 4930–4944, Sep. 2022, doi: 10.1109/TNNLS.2021.3063265.

[7] D. Gurevin, M. Shan, S. Huang, M. A. Hasan, C. Ding, and O. Khan, "PruneGNN: Algorithm-architecture pruning framework for graph neural network acceleration," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Edinburgh, U.K., Mar. 2024, pp. 108–123, doi: 10.1109/hpca57654.2024.00019.

[8] L. Liu, T. Liu, B. Jiang, and E. F. Y. Young, "Parmesan: Efficient partitioning and mapping flow for DNN training on general device topology," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 8, pp. 2426–2439, Aug. 2024, doi: 10.1109/TCAD.2024.3368970.

[9] Q. Zheng et al., "PIMulator-NN: An event-driven, cross-level simulation framework for processing-in-memory-based neural network accelerators," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 12, pp. 5464–5475, Dec. 2022, doi: 10.1109/TCAD.2022.3160947.

[10] H. Oh et al., "Energy-efficient in-memory binary neural network accelerator design based on 8T2C SRAM cell," *IEEE Solid-State Circuits Lett.*, vol. 5, pp. 70–73, 2022, doi: 10.1109/LSSC.2022.3161592.

[11] H. Wang, R. Liu, R. Dorrance, D. Dasalukunte, D. Lake, and B. Carlton, "A charge domain SRAM compute-in-memory macro with C-2C ladder-based 8-bit MAC unit in 22-nm FinFET process for edge inference," *IEEE J. Solid-State Circuits*, vol. 58, no. 4, pp. 1037–1050, Apr. 2023, doi: 10.1109/JSSC.2022.3232601.

[12] E. Choi et al., "A 133.6TOPS/W compute-in-memory SRAM macro with fully parallel one-step multi-bit computation," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2022, pp. 1–2, doi: 10.1109/CICC53496.2022.9772821.

[13] R. Sehgal, T. Thareja, S. Xie, C. Ni, and J. P. Kulkarni, "A bit-serial, compute-in-SRAM design featuring hybrid-integrating ADCs and input dependent binary scaled precharge eliminating DACs for energy-efficient DNN inference," *IEEE J. Solid-State Circuits*, vol. 58, no. 7, pp. 2109–2124, Jul. 2023, doi: 10.1109/JSSC.2023.3235210.

[14] H. Jeong, S. Kim, K. Park, J. Jung, and K. J. Lee, "A ternary neural network computing-in-memory processor with 16T1C bitcell architecture," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 5, pp. 1739–1743, May 2023, doi: 10.1109/TCSII.2023.3265064.

[15] M. E. Sinangil et al., "A 7-nm compute-in-memory SRAM macro supporting multi-bit input, weight and output and achieving 351 TOPS/W and 372.4 GOPS," *IEEE J. Solid-State Circuits*, vol. 56, no. 1, pp. 188–198, Jan. 2021, doi: 10.1109/JSSC.2020.3031290.

[16] D. Wang, C. T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy, and M. Seok, "DIMC: 2219TOPS/W 2569F2/b digital in-memory computing macro in 28nm based on approximate arithmetic hardware," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 266–268, doi: 10.1109/ISSCC42614.2022.9731659.

[17] C. He et al., "LSAC: A low-power adder tree for digital computing-in-memory by sparsity and approximate circuits co-design," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 2, pp. 852–856, Feb. 2024, doi: 10.1109/TCSII.2023.3304752.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:30:07 UTC from IEEE Xplore. Restrictions apply.

[18] S. Cai et al., "BASER: Bit-wise approximate compressor configurable in-SRAM-computing for energy-efficient neural network acceleration with data-aware weight remapping method," *Integr. Circuits Syst.*, vol. 1, no. 2, pp. 80–91, May 2024, doi: 10.23919/ics.2024.3419630.

[19] Y.-D. Chih et al., "16.4 An 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2021, pp. 252–254, doi: 10.1109/isscc42613.2021.9365766.

[20] A. Guo et al., "A 28nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 128–130, doi: 10.1109/ISSCC42615.2023.10067260.

[21] H. Kim, T. Yoo, T. T. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021, doi: 10.1109/JSSC.2021.3061508.

[22] Y. Ma et al., "DCIM-GCN: Digital computing-in-memory accelerator for graph convolutional network," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 6, pp. 2735–2748, Jun. 2024, doi: 10.1109/TCSI.2024.3384748.

[23] M.-G. Lin, J.-P. Wang, C.-Y. Chang, and A.-Y.-A. Wu, "Approximate adder tree design with sparsity-aware encoding and in-memory swapping for SRAM-based digital compute-in-memory macros," in *Proc. IEEE 6th Int. Conf. AI Circuits Syst. (AICAS)*, Abu Dhabi, United Arab Emirates, Apr. 2024, pp. 362–366, doi: 10.1109/AICAS59952.2024.10595967.

[24] H. Fujiwara et al., "34.4 A 3nm, 32.5TOPS/W, 55.0TOPS/mm^2 and 3.78Mb/mm^2 fully-digital compute-in-memory macro supporting INT12 × INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, Feb. 2024, pp. 572–574, doi: 10.1109/isscc49657.2024.10454556.

[25] X. Si et al., "A 28 nm 16-kb sign-extension-less digital-compute-in-memory macro with extension-friendly compute units and accuracy-adjustable adder-tree," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 11, pp. 2164–2168, Nov. 2024, doi: 10.1109/TVLSI.2024.3418888.

[26] Z. Lin, Y. Liu, Y. Wang, Y. Zhao, C. Peng, and X. Wu, "SRAM-based digital CIM macro for linear interpolation and MAC," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Singapore, May 2024, pp. 1–5, doi: 10.1109/iscas58744.2024.10558525.

[27] D. Shin, W. Choi, J. Park, and S. Ghosh, "Sensitivity-based error resilient techniques with heterogeneous multiply-accumulate unit for voltage scalable deep neural network accelerators," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 3, pp. 520–531, Sep. 2019, doi: 10.1109/JETCAS.2019.2933862.

[28] C. De la Parra, A. El-Yamany, T. Soliman, A. Kumar, N. Wehn, and A. Guntoro, "Exploiting resiliency for kernel-wise CNN approximation enabled by adaptive hardware design," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5, doi: 10.1109/ISCAS51556.2021.9401517.

[29] E. Trommer, B. Waschneck, and A. Kumar, "Combining gradients and probabilities for heterogeneous approximation of neural networks," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, San Diego, CA, USA, Oct. 2022, pp. 1–8.

[30] C. Yang and H. Liu, "Channel pruning based on convolutional neural network sensitivity," *Neurocomputing*, vol. 507, pp. 97–106, Oct. 2022, doi: 10.1016/j.neucom.2022.07.051.

[31] J. Chen et al., "AutoDCIM: An automated digital CIM compiler," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Jul. 2023, pp. 1–6, doi: 10.1109/dac56929.2023.10247976.

[32] C. Amarnath, M. Mejri, K. Ma, and A. Chatterjee, "Error resilience in deep neural networks using neuron gradient statistics," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 4, pp. 1149–1162, Apr. 2024, doi: 10.1109/TCAD.2023.3335144.

[33] H. Kim and A. A. D. Barrio, "A cost-efficient approximate dynamic ranged multiplication and approximation-aware training on convolutional neural networks," *IEEE Access*, vol. 9, pp. 135513–135525, 2021, doi: 10.1109/ACCESS.2021.3117148.

[34] M. S. Kim, A. A. Del Barrio, H. Kim, and N. Bagherzadeh, "The effects of approximate multiplication on convolutional neural networks," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 2, pp. 904–916, Apr. 2022, doi: 10.1109/TETC.2021.3050989.

[35] H. Jiang, X. Peng, S. Huang, and S. Yu, "CIMAT: A compute-in-memory architecture for on-chip training based on transpose SRAM arrays," *IEEE Trans. Comput.*, vol. 69, no. 7, pp. 944–954, Jul. 2020, doi: 10.1109/TC.2020.2980533.

[36] A. Kneip, M. Lefebvre, J. Verecken, and D. Bol, "IMPACT: A 1-to-4b 813-TOPS/W 22-nm FD-SOI Compute-in-Memory CNN accelerator featuring a 4.2-POPS/W 146-TOPS/mm^2 CIM-SRAM with multi-bit analog batch-normalization," *IEEE J. Solid-State Circuits*, vol. 58, no. 7, pp. 1871–1884, Jul. 2023, doi: 10.1109/JSSC.2023.3269098.

[37] F. Frustaci, S. Perri, P. Corsonello, and M. Alioto, "Energy-quality scalable adders based on nonzeroing bit truncation," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 4, pp. 964–968, Apr. 2019, doi: 10.1109/TVLSI.2018.2881326.

[38] H. Diao et al., "A multiply-less approximate SRAM compute-in-memory macro for neural-network inference," *IEEE J. Solid-State Circuits*, early access, Aug. 5, 2024, doi: 10.1109/JSSC.2024.3433417.



Kyeongho Lee (Member, IEEE) received the Ph.D. degree in electrical and computer engineering from Korea University, Seoul, South Korea, in 2024. He joined the Library Development Team, Foundry Business of Samsung Electronics, Hwaseong-si, South Korea, in 2024. His research interests include cost-efficient design techniques for SRAM-based compute-in-memory and embedded memory designs in advanced technologies.



Hyeyeong Lee (Graduate Student Member, IEEE) received the B.S. and M.S. degrees in electronics engineering from Korea University, Seoul, South Korea, in 2022 and 2024, respectively. She joined with the Library Development Team, Foundry Business of Samsung Electronics, Hwaseong-si, South Korea, in 2024. Her current research interests include the neural network accelerator and embedded memory designs in advanced technologies.



Jongsun Park (Senior Member, IEEE) received the B.S. degree in electronics engineering from Korea University, Seoul, South Korea, in 1998, and the M.S. and Ph.D. degrees in electrical and computer engineering from Purdue University, West Lafayette, IN, USA, in 2000 and 2005, respectively.

He joined the Electrical Engineering Faculty, Korea University, in 2008. He was with the Digital Radio Processor System Design Group, Texas Instruments, Dallas, TX, USA, in 2002. From 2005 to 2008, he was with the Signal Processing Technology Group, Marvell Semiconductor Inc., Santa Clara, CA, USA. His research interests include variation-tolerant, low-power, high-performance VLSI architectures, and circuit designs for digital signal processing and digital communications.

Dr. Park is the Chair-Elect of the Circuits and Systems for Communications Technical Committee of the IEEE Circuits and Systems Society. He served as an Associate Editor for IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR ARTIFICIAL INTELLIGENCE and a Guest Editor for IEEE JOURNAL ON EMERGING AND SELECTED TOPICS IN CIRCUITS AND SYSTEMS. He has also served on the technical program committees of various IEEE/ACM conferences, including DAC, ICCAD, ISLPED, ISCAS, ASP-DAC, HOST, VLSI-SoC, and AICAS. He is an Associate Editor of IEEE SOLID-STATE CIRCUITS LETTERS and *Journal of Low Power Electronics and Applications*.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:30:07 UTC from IEEE Xplore. Restrictions apply.